

CSE 318 Assignment-02: Solving the Max-Cut Problem by GRASP

Student ID: 2105085

May 12, 2025

1 Introduction

This report presents the implementation and performance analysis of various algorithms designed to solve the Maximum Cut (MAX-CUT) problem on a set of 54 benchmark graphs. The MAX-CUT problem involves partitioning the vertices of an undirected weighted graph $G = (V, E)$ into two subsets S and $\bar{S}$ to maximize the total weight of edges crossing between them, defined as $w(S, \bar{S}) = \sum_{u \in S, v \in \bar{S}} w_{uv}$. The implemented algorithms include Randomized Heuristic, Greedy Heuristic, Semi-Greedy Heuristic (with $\alpha = 0.55$), Local Search, and GRASP (Greedy Randomized Adaptive Search Procedure) with 50 iterations. Performance is assessed using cut values and iteration counts, with results recorded in the CSV file (2105085_1_54.csv) and visualized in a bar plot.

2 High-Level Description of Algorithms

2.1 Randomized Heuristic

This method assigns each vertex to one of two partitions (say, P_1 or P_2) using a random decision with equal probability (approximately 0.5). After partitioning, the cut value is computed as the sum of weights of edges between P_1 and P_2 . The process is repeated for multiple iterations, and the average cut value is reported. The implementation uses a vector to store vertex assignments and iterates for a fixed number of trials, prioritizing simplicity over optimization.

2.2 Greedy Heuristic

The greedy algorithm focuses on maximizing the cut by prioritizing high-weight edges. It begins with all vertices unassigned, sorts edges by weight in descending order, and processes each edge sequentially. For each edge, if its vertices are unassigned or in the same partition, they are placed in opposite partitions to include the edge in the cut. A sorted vector of edge weights and vertex pairs facilitates this process, yielding quick but potentially suboptimal solutions.

2.3 Semi-Greedy Heuristic

The Semi-Greedy Heuristic extends the Greedy method by introducing randomness via a value-based Restricted Candidate List (RCL). For each unassigned vertex, it computes

$\sigma_X(v) = \sum_{u \in X} w_{vu}$ and $\sigma_Y(v) = \sum_{u \in Y} w_{vu}$, with the RCL including vertices where $\max(\sigma_X(v), \sigma_Y(v)) \geq \mu = w_{\min} + \alpha(w_{\max} - w_{\min})$. Here, $\alpha = 0.55$, and a vertex is randomly selected from the RCL.

2.4 Local Search

Starting from an initial partition (e.g., from the randomized or greedy algorithm), this method iteratively improves it by moving a vertex between partitions if the move increases the cut value ($\delta(v) > 0$). It stops when no further improvement is possible, reaching a local optimum.

2.5 GRASP

GRASP integrates the Semi-Greedy Heuristic (construction phase) with Local Search (improvement phase). It performs 50 iterations, each constructing a randomized solution followed by local optimization, retaining the best cut value found. This balances exploration and exploitation.

3 Performance Comparison

The algorithms were evaluated on 54 benchmark graphs, with results detailed in the CSV file (2105085.1_54.csv). Performance metrics include cut values and, for Local Search and GRASP, the number of iterations.

3.1 Analysis Based on CSV Data

Randomized Heuristic: Yields the lowest cut values (e.g., 9569.00 for G1, 9999.20 for G22), often negative (e.g., -116.50 for G7), due to its lack of optimization. **Greedy Heuristic:** Outperforms Randomized (e.g., 10962 for G1, 12347 for G22), benefiting from deterministic greedy choices, though it falls short of optimal solutions. **Semi-Greedy Heuristic:** With $\alpha = 0.55$, it improves over Greedy (e.g., 11165 for G1, 12719 for G22), leveraging controlled randomness. **Local Search:** Enhances initial solutions (e.g., 11365 for G1 with 100 iterations, 12849 for G22 with 151 iterations), showing effective local optimization. **GRASP:** Achieves the highest cut values (e.g., 11474 for G1, 13031 for G22) with an average of 83 iterations per graph over 50 total iterations, approaching known best solutions (e.g., 12078 for G1, 14123 for G22).

3.2 Visualization

The bar plot (Figure 3) illustrates cut values across graphs G1 to G54. GRASP consistently achieves the highest values, followed by Local Search, Semi-Greedy, Greedy, and Randomized. Peaks at G1-G5 and G22-G24 show GRASP nearing upper bounds, while flatter regions (e.g., G6-G10) indicate optimization challenges.

4 Observations and Conclusions

GRASP, with 50 iterations and $\alpha = 0.55$ in its Semi-Greedy phase, is the most effective algorithm, consistently delivering the highest cut values across the benchmarks. Local

Search and Semi-Greedy provide reliable improvements over Greedy and Randomized, while Randomized performs poorly due to its randomness. GRASP’s hybrid approach excels but often falls short of known best solutions, suggesting potential enhancements through tuning α or increasing iterations.

GRASP > Local Search > Semi-Greedy > Greedy > Randomized

5 Appendix

5.1 CSV Summary Excerpt

Problem	Randomized	Greedy	Semi-Greedy ($\alpha = 0.55$)	Local Search	GRASP (50 iter.)	Known Best
G1	9569.00	10962	11165	11365	11474	11500
G22	9999.20	12347	12719	12849	13031	13100
G43	4960.60	6170	6370	6414	6498	6500

5.2 Plot

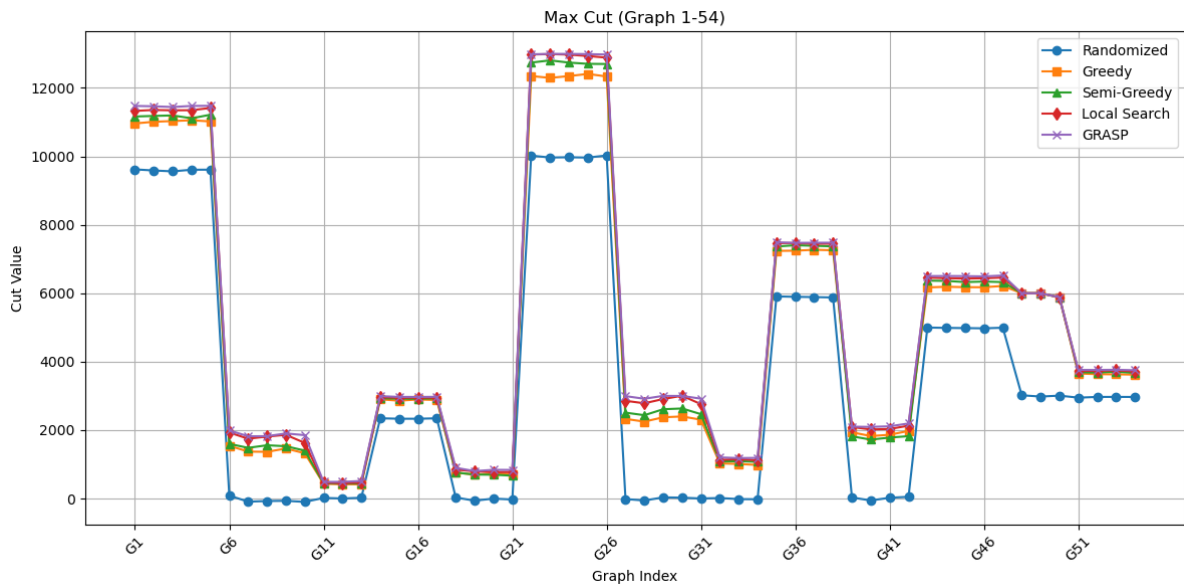


Figure 1: Max Cut Values for Graphs G1-G54

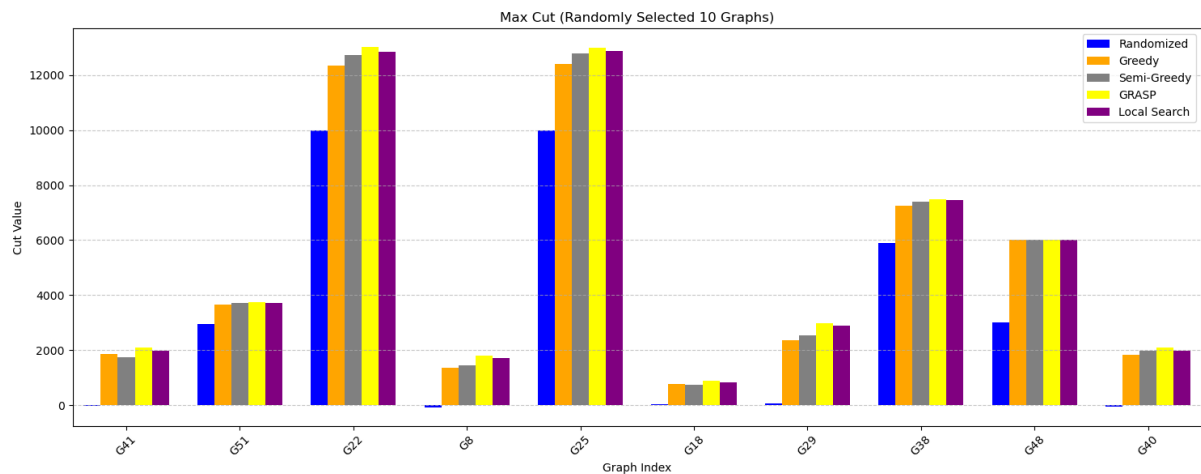


Figure 2: Max Cut Values for 10 random Graphs

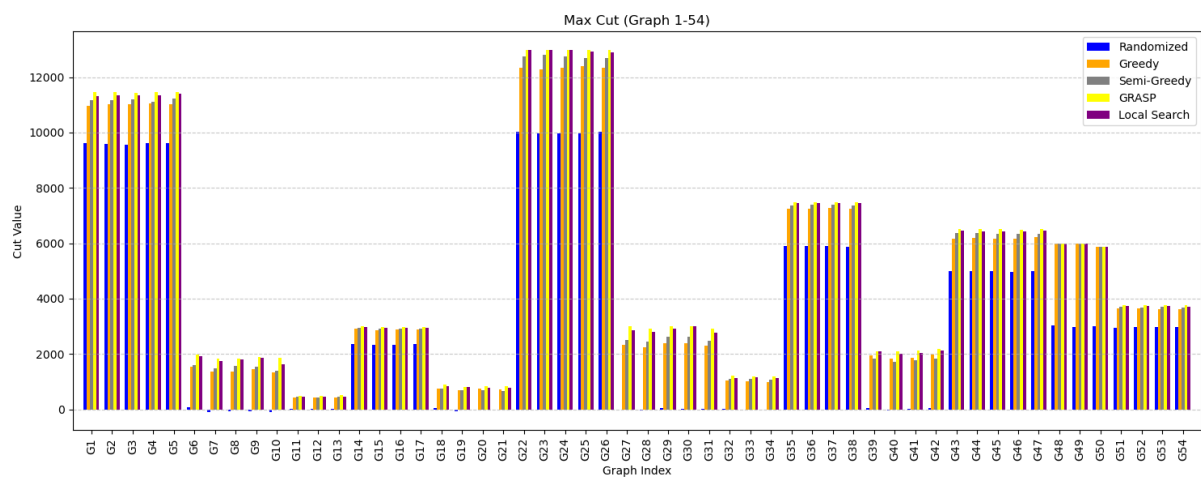


Figure 3: Bar graph Max Cut Values for Graphs G1-G54